

Apache Iceberg

Parquet Module Design Document

Architecture Design / Design Patterns / Call Chain Analysis

Project	Apache Iceberg v1.9.2
Module	iceberg-parquet
Source Files	51 Java files (~9000 lines)
Document	Architecture / Flow / Sequence / Pattern / Call Chain

Table of Contents

1 Module Overview

2 Architecture Design

2.1 Layered Architecture

2.2 Core Class Inventory

3 Flow Diagrams

3.1 Parquet Read Flow

3.2 Parquet Write Flow

4 Sequence Diagrams

4.1 Three-Level Row Group Filtering

5 Design Pattern Analysis

5.1 Facade Pattern - Parquet.java

5.2 Builder Pattern - WriteBuilder/ReadBuilder

5.3 Visitor Pattern - TypeVisitor Family

5.4 Strategy Pattern - Filter Chain

5.5 Template Method - BasePageIterator

5.6 Adapter Pattern - ParquetIO

6 Key Call Chains

6.1 Data Read Call Chain

6.2 Data Write Call Chain

6.3 Schema Evolution Call Chain

6.4 Metrics Collection Call Chain

7 Variant Type Support (v1.9 New)

8 Summary

1. Module Overview

The iceberg-parquet module is the bridge between Apache Iceberg table format and Apache Parquet file format. It is one of the most critical modules in the Iceberg project, responsible for all Parquet file read/write operations, schema conversion, predicate pushdown, metrics collection, and IO adaptation.

The module provides a clean abstraction layer that enables Iceberg to work with Parquet files without leaking Parquet-specific details to the upper API or engine-specific integration layers (Spark, Flink, etc.). This module is engine-agnostic by design.

Core Responsibilities

Key Responsibilities:

1. Schema Conversion: Bidirectional mapping between Iceberg Schema and Parquet MessageType
2. File Read/Write: Custom Parquet reader/writer with Iceberg-specific optimizations
3. Predicate Pushdown: Three-level row group filtering (stats/dict/bloom)
4. Metrics Collection: Extract column statistics for table metadata
5. IO Adaptation: Bridge Iceberg IO API to Parquet IO API
6. Variant Support: Shredded variant column read/write (new in v1.9)

2. Architecture Design

2.1 Layered Architecture

The Parquet module follows a clear 5-layer architecture. Each layer has well-defined responsibilities and communicates only with adjacent layers through clean interfaces.

Iceberg Parquet Module Architecture

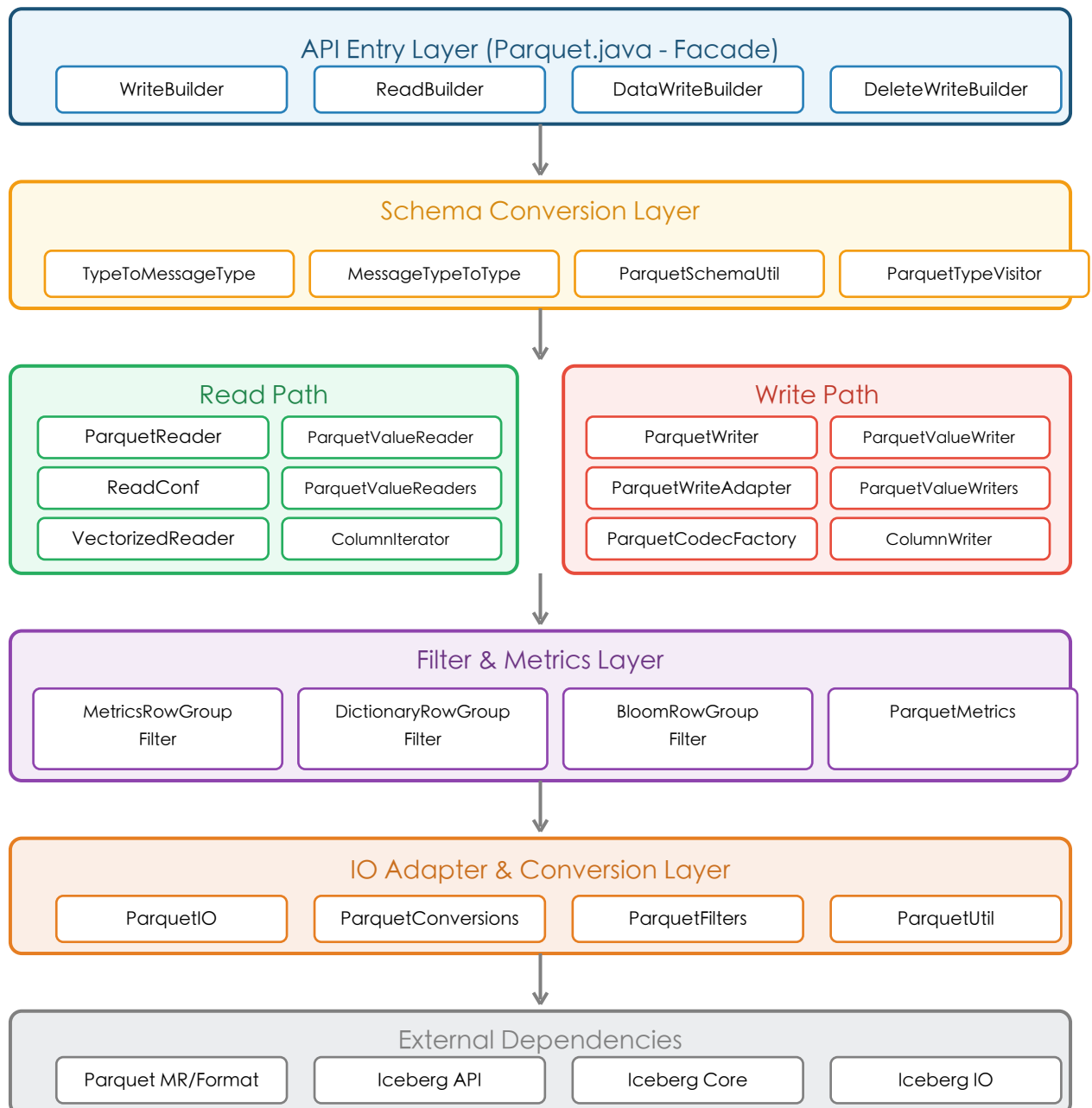


Figure 1: Iceberg Parquet Module Architecture

Layer 1 - API Entry (Facade): `Parquet.java` serves as the single entry point, providing Builder APIs for read, write, data-write, and delete-write operations. All engine integrations interact

with this layer exclusively.

Layer 2 - Schema Conversion: Handles bidirectional schema mapping between Iceberg types and Parquet MessageType. Includes column pruning, ID management, and name mapping support.

Layer 3 - Read/Write Core: The core I/O engine. Read path uses ParquetReader + ParquetValueReaders for row-by-row reading. Write path uses ParquetWriter + ParquetValueWriters with adaptive row group sizing.

Layer 4 - Filter & Metrics: Implements three-level predicate pushdown and column statistics extraction. Each filter operates on different data structures for progressive elimination.

Layer 5 - IO Adapter: Bridges Iceberg IO API to Parquet IO API, handles type conversions, and provides utility functions.

2.2 Core Class Inventory

Class	Lines	Role	Design Pattern
Parquet.java	1641	Module entry point, all Builder APIs	Facade + Builder
ParquetReader.java	178	Custom file reader, row-by-row iteration	Iterator
ParquetWriter.java	269	Custom file writer, adaptive row groups	Template Method
ParquetValueReaders.java	1222	All type-specific value reader impls	Strategy + Composite
ParquetValueWriters.java	733	All type-specific value writer impls	Strategy + Composite
ReadConf.java	224	Read configuration and initialization	Configuration Object
ParquetSchemaUtil.java	285	Schema utility functions	Utility
ParquetTypeVisitor.java	275	Parquet type tree traversal	Visitor
TypeWithSchemaVisitor.java	264	Dual-tree traversal (Iceberg+Parquet)	Visitor
TypeToMessageType.java	292	Iceberg Schema -> Parquet MessageType	Visitor
MessageTypeToType.java	265	Parquet MessageType -> Iceberg Schema	Visitor
ParquetMetrics.java	648	Extract column statistics from files	Visitor
ParquetMetricsRowGroupFilter	599	Stats-based row group filter	Strategy
ParquetDictRowGroupFilter	481	Dictionary-based row group filter	Strategy
ParquetBloomRowGroupFilter	353	Bloom filter-based row group filter	Strategy
ParquetIO.java	267	IO API adaptation	Adapter
ColumnIterator.java	164	Column-level data iteration	Iterator
ColumnWriter.java	133	Column-level data writing	Iterator
ParquetVariantVisitor.java	287	Variant type tree traversal	Visitor

3. Flow Diagrams

3.1 Parquet Read Flow

The read path follows a pipeline architecture where data flows through schema matching, row group filtering, value reader creation, and record assembly stages.

Parquet Read Flow

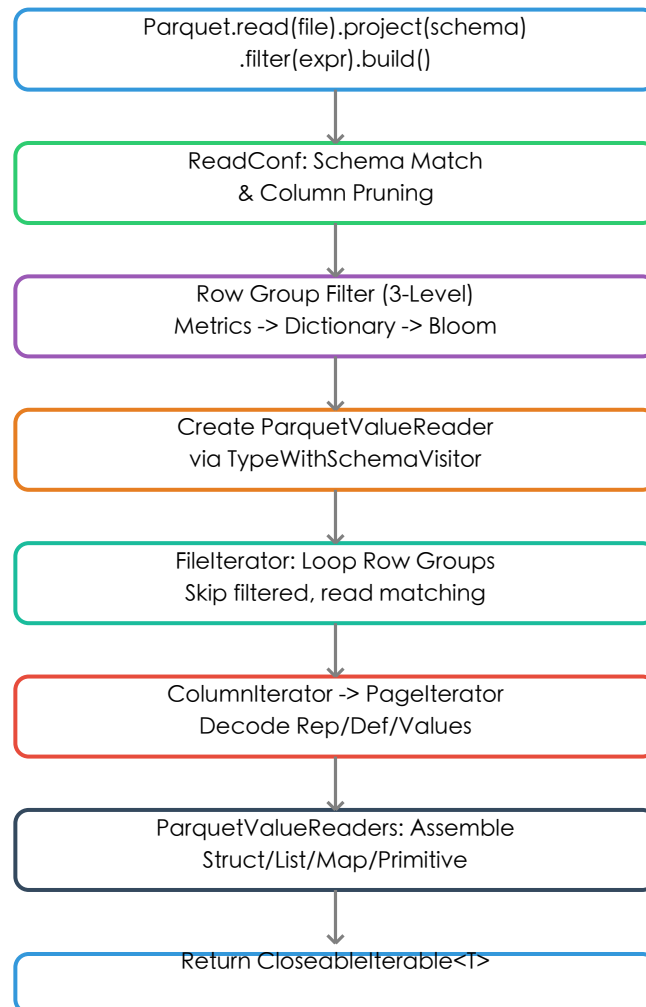


Figure 2: Parquet Read Flow

Step 1 - API Entry: User calls `Parquet.read(file).project(schema).filter(expr).build()`, which creates a `ParquetReader` with all configurations captured.

Step 2 - Schema Match: `ReadConf` handles three scenarios: (a) file has Iceberg IDs, (b) name mapping available, (c) fallback ID assignment. Then performs column pruning.

Step 3 - Row Group Filter: Three-level progressive filtering. `MetricsFilter` uses min/max/null-count statistics; `DictionaryFilter` checks dictionary-encoded values; `BloomFilter` probes bloom filter pages. Each level produces a `boolean[] shouldSkip` array that is merged.

Step 4 - Reader Creation: `TypeWithSchemaVisitor` simultaneously walks the Iceberg type tree and Parquet schema tree, creating the appropriate `ParquetValueReader` for each column.

Step 5-7 - Data Reading: `FileIterator` loops through non-skipped row groups. For each row group, `columnIterator` reads pages, `PageIterator` decodes repetition/definition levels and values, and type-specific readers assemble the final records.

3.2 Parquet Write Flow

The write path converts Iceberg records to Parquet format with adaptive row group sizing and comprehensive metrics collection.

Parquet Write Flow

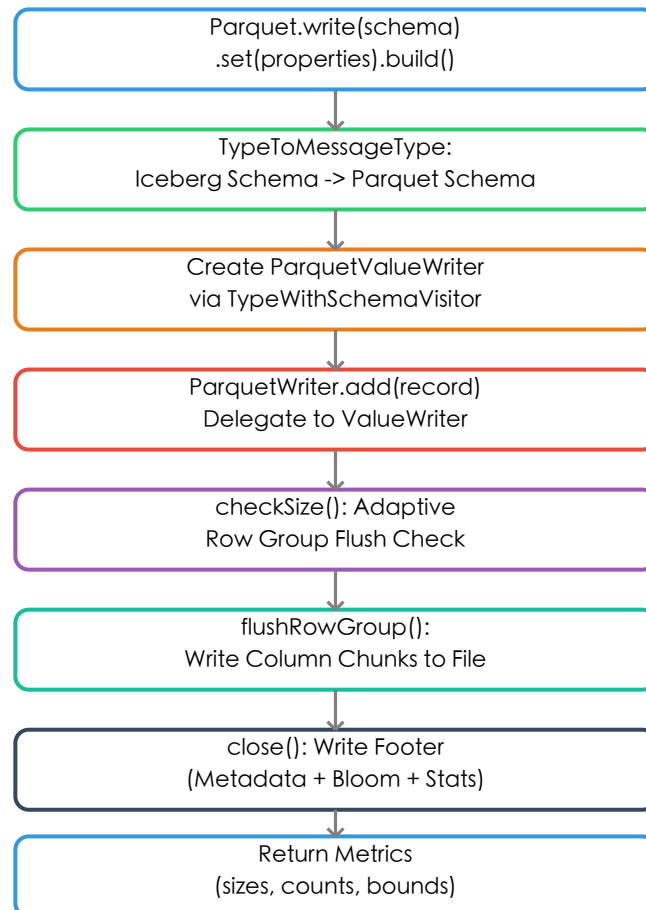


Figure 3: Parquet Write Flow

Adaptive Row Group Sizing: The `checkSize()` method in `ParquetWriter` implements a smart algorithm: it estimates average record size from the buffered data, then calculates how many records until the target row group size (default 128MB) is reached. This avoids checking size after every record, dramatically reducing overhead. The next check count starts at `min(max(recordsWritten/2, 100), 10000)`.

Lazy Initialization: `ParquetWriter` uses `ensureWriterInitialized()` to delay creating the underlying `ParquetFileWriter` until the first record is actually written. This prevents creating empty files when no data needs to be written.

4. Sequence Diagrams

4.1 Three-Level Row Group Filtering

Row group filtering is one of the most performance-critical paths in Iceberg. The three-level filtering mechanism progressively eliminates row groups that cannot contain matching data, significantly reducing I/O.

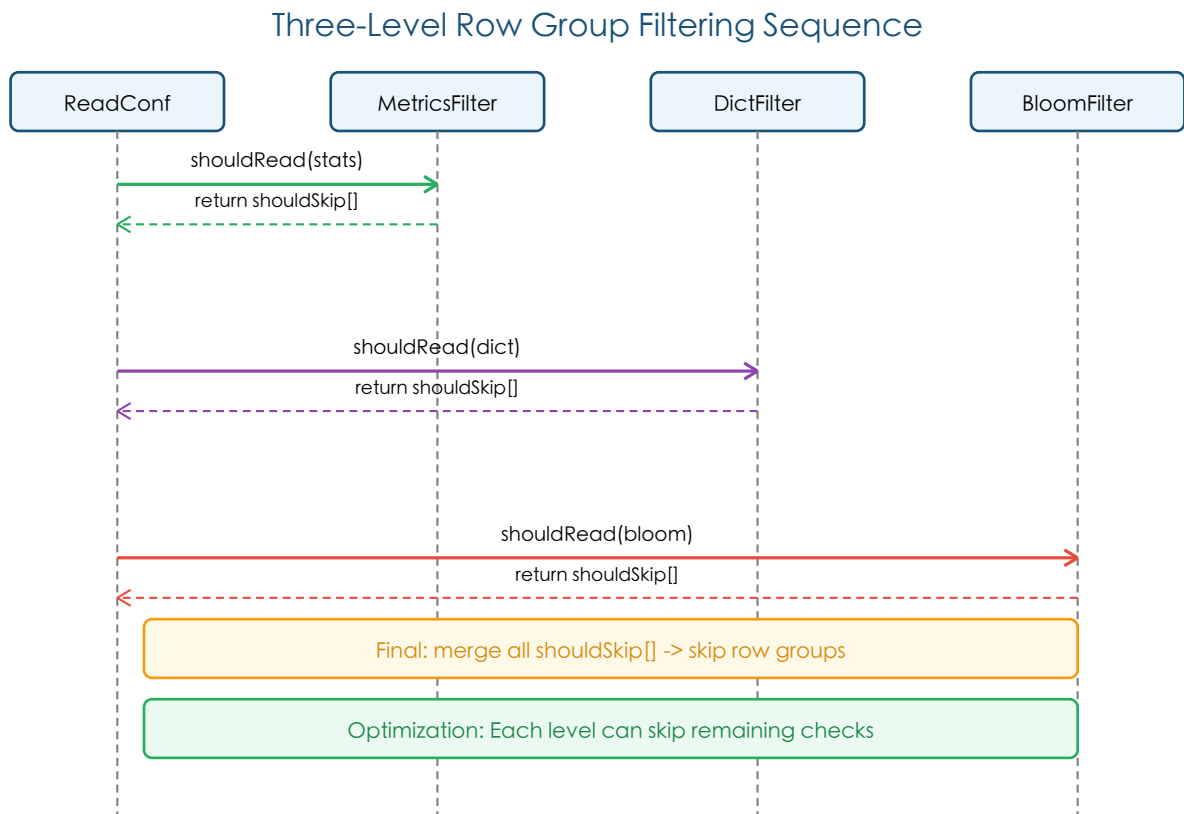


Figure 4: Three-Level Row Group Filtering Sequence

Level 1 - Metrics Filter (ParquetMetricsRowGroupFilter): Evaluates expressions against column statistics (min, max, null count, NaN count). This is the cheapest filter as statistics are stored in the footer metadata.

Level 2 - Dictionary Filter (ParquetDictionaryRowGroupFilter): For dictionary-encoded columns, reads the dictionary page and checks if any dictionary value matches the predicate. More precise than statistics for equality and IN predicates.

Level 3 - Bloom Filter (ParquetBloomRowGroupFilter): Probes the bloom filter for equality checks. Bloom filters can definitively say a value is NOT present (no false negatives) but may have false positives. Only effective for EQ and IN predicates.

Performance Insight

Performance Impact: In production workloads with selective predicates, the three-level filtering can skip 90%+ of row groups, reducing I/O by orders of magnitude. The cost of each level is minimal compared to reading actual data pages.

5. Design Pattern Analysis

5.1 Facade Pattern - Parquet.java

`Parquet.java` (1641 lines) is the Facade of the entire module. It encapsulates the complexity of 50+ internal classes behind four clean Builder APIs:

Builder	Purpose	Key Methods
<code>WriteBuilder</code>	Build file appender	<code>schema()</code> , <code>set()</code> , <code>overwrite()</code> , <code>metricsConfig()</code>
<code>ReadBuilder</code>	Build file reader	<code>project()</code> , <code>filter()</code> , <code>reuseContainers()</code> , <code>split()</code>
<code>DataWriteBuilder</code>	Build data writer	<code>forTable()</code> , <code>withSpec()</code> , <code>withPartition()</code>
<code>DeleteWriteBuilder</code>	Build delete writer	<code>equalityDelete()</code> , <code>positionDelete()</code>

The Facade pattern here serves two critical purposes: (1) It provides a fluent API that guides users through complex configuration, and (2) it hides the internal wiring between schema conversion, reader/writer creation, filter setup, and IO adaptation.

5.2 Builder Pattern - `WriteBuilder` / `ReadBuilder`

Each Builder in `Parquet.java` implements the Builder pattern with method chaining. The `writeBuilder` manages 20+ configuration parameters through fluent setter methods. Internally, it uses a `context` inner class to manage two sets of configurations: one for data writing and one for delete writing.

The `build()` method orchestrates the entire construction process: schema conversion via `TypeToMessageType`, value writer/reader creation via `TypeWithSchemaVisitor`, and the final assembly of `ParquetWriter` Or `ParquetReader`.

5.3 Visitor Pattern - `TypeVisitor` Family

The Visitor pattern is the most extensively used pattern in this module, with three visitor hierarchies serving different purposes:

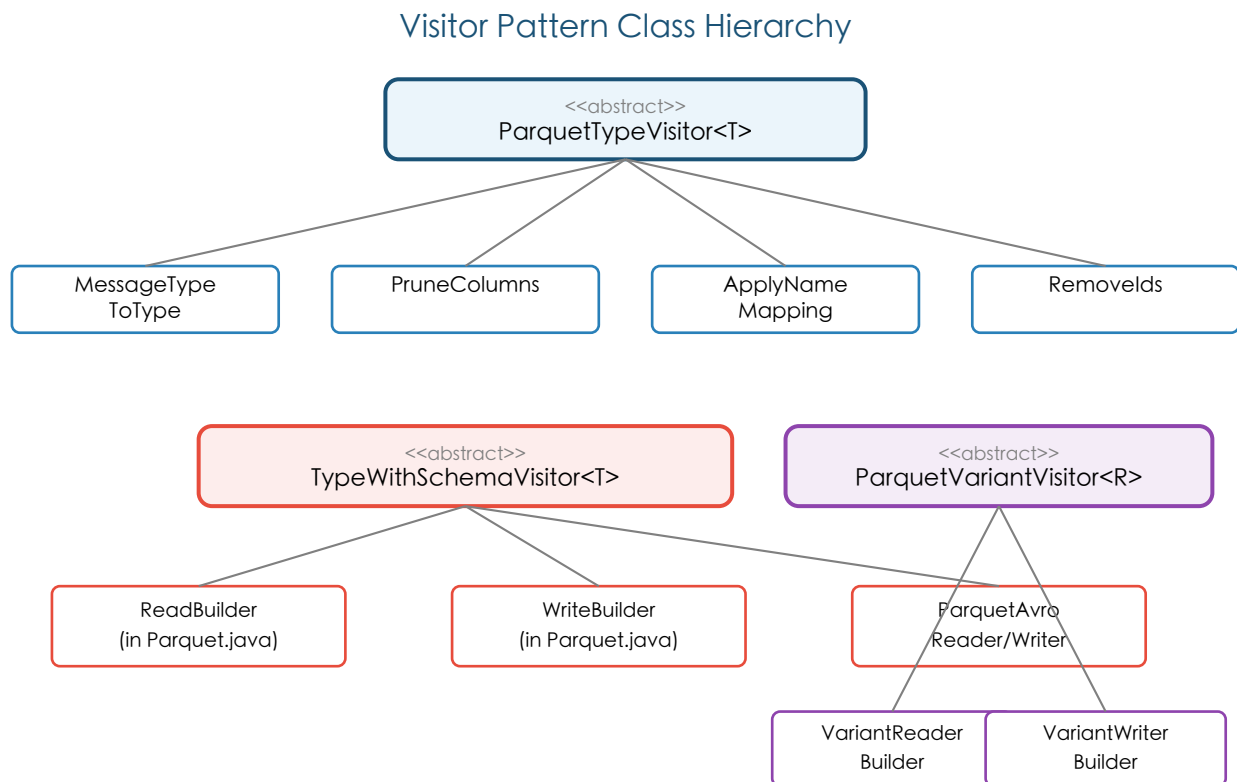


Figure 5: Visitor Pattern Class Hierarchy

ParquetTypeVisitor<T> - Traverses a single Parquet MessageType tree. Used by MessageTypeToType (Parquet->Iceberg conversion), PruneColumns (column pruning), ApplyNameMapping (name-based ID assignment), and RemoveIds (ID removal). Handles complex list/map encoding with 2-level and 3-level detection.

TypeWithSchemaVisitor<T> - Simultaneously traverses both Iceberg Type and Parquet Type trees. This dual-tree walk is essential for creating type-appropriate readers/writers. The visitor matches fields by Parquet field ID to find corresponding Iceberg NestedField, handling schema evolution gracefully.

ParquetVariantVisitor<R> - Traverses variant-specific Parquet structures (metadata, value, typed_value, shredded objects/arrays). Used by VariantReaderBuilder and VariantWriterBuilder for Variant column support.

5.4 Strategy Pattern - Filter Chain

The three row group filters implement the Strategy pattern. Each filter encapsulates a different algorithm for evaluating whether a row group can be skipped:

Strategy	Data Source	Applicable Predicates	Precision
MetricsRowGroupFilter	Column statistics (footer)	All comparison ops, IN, startsWith	Coarse (range-based)
DictionaryRowGroupFilter	Dictionary pages	eq, notEq, in, notIn, startsWith	Fine (exact values)
BloomRowGroupFilter	Bloom filter pages	eq, in only	Probabilistic (no FN)

All three filters share the same interface: `shouldRead(MessageType, BlockMetaData, ...)` returning a boolean. They are composed in `ReadConf` as a filter chain where each level narrows the set of row groups to read.

5.5 Template Method - BasePageIterator

`BasePageIterator` defines the Template Method pattern for page decoding. The template method `initFromPage()` defines the invariant steps: (1) read repetition levels, (2) read definition levels, (3) init data reader. Subclasses provide concrete implementations for each step.

Abstract Method	PageIterator (Row-by-Row)	VectorizedPageIterator (Batched)
<code>initDataReader()</code>	ValuesReader from encoding	ValuesAsBytesReader for batch
<code>initDefinitionLevels()</code>	ValuesReaderIntIterator	RLE batch decode

The column-level iteration hierarchy follows the same principle: `BaseColumnIterator` defines `setPageSource()` and `advance()` template methods, with `columnIterator` (row-by-row) and vectorized variants as concrete implementations.

5.6 Adapter Pattern - ParquetIO

`ParquetIO` implements the Adapter pattern to bridge the gap between Iceberg IO API (`InputFile`, `OutputFile`, `SeekableInputStream`) and Parquet IO API (`org.apache.parquet.io.InputFile`, etc.).

Iceberg API	Parquet API	Adapter Class
<code>o.a.iceberg.io.InputFile</code>	<code>o.a.parquet.io.InputFile</code>	<code>ParquetInputFile</code>
<code>o.a.iceberg.io.OutputFile</code>	<code>o.a.parquet.io.OutputFile</code>	<code>ParquetOutputFile</code>
<code>SeekableInputStream</code>	<code>SeekableInputStream</code>	<code>ParquetInputStreamAdapter</code>

Iceberg API	Parquet API	Adapter Class
PositionOutputStream	PositionOutputStream	ParquetOutputStreamAdapter

For Hadoop-backed files, `ParquetIO` detects and directly unwraps the Hadoop file objects for optimal performance, avoiding double-wrapping overhead.

6. Key Call Chains

6.1 Data Read Call Chain

Complete call chain for reading a Parquet file with projection and filtering:

Step	Class.Method	Description
1	Parquet.read(InputFile)	Create ReadBuilder (Facade entry)
2	ReadBuilder.project(Schema)	Set expected projection schema
3	ReadBuilder.filter(Expression)	Set pushdown filter expression
4	ReadBuilder.build() -> createReaderAs	Assemble ParquetReader with all configs
5	ParquetReader.init()	Lazy init: create ReadConf
6	ReadConf.<init>()	Open ParquetFileReader, match schemas
7	ParquetSchemaUtil.pruneColumns()	Prune file schema to projection
8	ReadConf: MetricsRowGroupFilter	Filter row groups by column stats
9	ReadConf: DictionaryRowGroupFilter	Filter by dictionary values
10	ReadConf: BloomRowGroupFilter	Filter by bloom filter
11	TypeWithSchemaVisitor.visit()	Create ParquetValueReader tree
12	FileIterator.next()	Advance to next non-skipped row group
13	ColumnIterator.setPageSource()	Bind to row group page store
14	PageIterator.setPage(DataPage)	Decode rep/def levels + values
15	ParquetValueReaders.*Reader.readAs	Assemble typed record from columns

6.2 Data Write Call Chain

Complete call chain for writing records to a Parquet file:

Step	Class.Method	Description
1	Parquet.write(Schema)	Create WriteBuilder (Facade entry)
2	WriteBuilder.set(properties)	Configure compression, row group size, etc.
3	WriteBuilder.build() -> createWriterFunc	Create writer function
4	TypeToMessageType.convert()	Convert Iceberg Schema to Parquet MessageType
5	TypeWithSchemaVisitor.visit()	Create ParquetValueWriter tree
6	ParquetWriter.<init>()	Initialize with lazy file creation

Step	Class.Method	Description
7	ParquetWriter.add(record)	Write one record
8	ensureWriterInitialized()	Create ParquetFileWriter on first write
9	ParquetValueWriter.write(rl, value)	Decompose record to column values
10	ColumnWriter/TripleWriter.write()	Write rep/def levels + value to column
11	checkSize() -> flushRowGroup()	Adaptive size check, flush when full
12	close() -> flushRowGroup(true)	Final flush + write footer metadata
13	ParquetWriter.metrics()	Return Metrics (sizes, counts, bounds)

6.3 Schema Evolution Call Chain

When reading a Parquet file whose schema does not exactly match the current table schema, the schema evolution chain handles three scenarios:

Step	Class.Method	Description
1	ReadConf: check hasIds(fileSchema)	Check if file schema has Iceberg field IDs
2a	If hasIds: use file schema directly	Best case: file was written by Iceberg
2b	If nameMapping != null: applyNameMapping()	Apply field names to Iceberg IDs via NameMapping
2c	Else: addFallbackIds()	Assign sequential IDs based on position
3	PruneColumns.visit(fileSchema, expectedSchema)	Remove columns not in projection
4	TypeWithSchemaVisitor: match by field ID	Join Iceberg fields with Parquet fields by ID
5	Handle missing fields: create NullReadColumns	Columns added after file was written -> null
6	Handle extra fields: skip column	Columns deleted after file was written -> ignore

6.4 Metrics Collection Call Chain

Metrics are collected both during writing (from ParquetValueWriters) and from file footer metadata:

Step	Class.Method	Description
1a	ParquetWriter.close()	After writing, collect writer-side metrics
1b	ParquetUtil.fileMetrics(file)	Or read metrics from existing file footer
2	ParquetMetrics.metrics()	Static method: extract from ParquetMetadata
3	MetricsVisitor (TypeWithSchemaVisitor)	Walk schema to build FieldMetrics per column
4	Aggregate per BlockMetaData	Sum sizes, counts across all row groups
5	Apply MetricsConfig truncation	Truncate string/binary bounds by configured length
6	Return Metrics object	Contains sizes, counts, null counts, NaN, bounds maps

Metrics Merge Strategy

The `FieldMetrics` from writers (tracking actual written values with NaN handling) are merged with footer statistics during `ParquetWriter.close()`. Writer metrics take precedence for NaN counts and float/double bounds, since Parquet statistics may not correctly handle NaN ordering.

7. Variant Type Support (v1.9 New)

Iceberg v1.9 introduces the Variant data type with Parquet shredding support. This enables semi-structured data (JSON-like) to be stored in Parquet with optimized columnar encoding. The Parquet module adds 8 new classes specifically for Variant handling.

Class	Role	Description
ParquetVariantVisitor	Visitor	Traverse variant Parquet schema (metadata/value/typed_value)
VariantReaderBuilder	Builder	Create ParquetValueReader tree for variant columns
VariantWriterBuilder	Builder	Create ParquetValueWriter tree for variant columns
ParquetVariantReaders	Factory	Variant-specific reader implementations (serialized/shredded/array/object)
ParquetVariantWriters	Factory	Variant-specific writer implementations
ParquetVariantUtil	Utility	Type conversion, serialization, metrics, schema production
VariantShreddingFunction	SPI	Function interface for producing shredded Parquet types
ParquetFormatModel	Model	Variant stats and metrics model for Parquet format

Variant Shredding: A Variant column in Parquet is stored as a struct with three fields: `metadata` (binary, required), `value` (binary, optional), and `typed_value` (optional, type-specific). When a Variant consistently contains data of a specific type (e.g., always an integer), the `typed_value` field stores it in native Parquet encoding (e.g., INT32) for efficient columnar access.

Object Shredding: For Variant objects with known fields, each field can be independently shredded into its own `value/typed_value` pair. The `ShreddedObjectReader` and `ShreddedObjectWriter` handle merging partially shredded objects with their serialized fallback.

8. Summary

The Apache Iceberg Parquet module is a masterfully designed component that demonstrates several key software engineering principles:

Principle	Implementation
Separation of Concerns	Five clean architectural layers with distinct responsibilities
Single Entry Point	Parquet.java Facade encapsulates 50+ internal classes
Progressive Optimization	Three-level filter chain: stats -> dictionary -> bloom
Schema Evolution	ID-based field matching with fallback strategies
Lazy Evaluation	Lazy file creation, lazy ReadConf init, lazy page reading
Extensibility	Visitor pattern enables new type support (Variant) without modifying core
Performance	Adaptive row group sizing, container reuse, vectorized reading
Abstraction	IO adapter layer isolates Parquet API from Iceberg API

Key Takeaway

The module processes ~9000 lines of carefully crafted Java code across 51 files. It employs 6 major design patterns (Facade, Builder, Visitor, Strategy, Template Method, Adapter) and provides comprehensive support for the full Parquet file lifecycle: schema conversion, data writing with adaptive sizing, three-level predicate pushdown, metrics collection, and the cutting-edge Variant type shredding. This design achieves a remarkable balance between performance, extensibility, and maintainability.